26.3 Webhook 与 Gmail Pub/Sub

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~220k tokens，输出 ~5k tokens（本节）

Cron 系统让 Agent 可以按时间计划执行任务。但有些场景不是按时间触发的——“收到一封新邮件时自动处理”、“外部系统发来 Webhook 时触发 Agent”。这一节看看 OpenClaw 的 Webhook 触发器和 Gmail Pub/Sub 集成是怎么做的。

26.3.1 Webhook 触发器

概念：HTTP 触发的 Agent 执行

Webhook 允许外部系统通过 HTTP 请求触发 Agent 执行。例如：

- GitHub 推送代码时通知 Agent 执行代码审查
- 监控系统检测到异常时让 Agent 生成分析报告
- IoT 设备发送传感器数据让 Agent 处理

OpenClaw 在 Gateway 的 HTTP 层提供 Webhook 端点，路径默认为 `/hooks` 。

配置与认证

Webhook 需要在配置文件中显式启用：

```yaml
hooks:
  enabled: true
  token: "my-secret-webhook-token"  # 必须设置
  path: "/hooks"                     # 默认值
  maxBodyBytes: 262144               # 256KB，默认值
```

```typescript
// src/gateway/hooks.ts — resolveHooksConfig

export function resolveHooksConfig(cfg: OpenClawConfig): HooksConfigResolved | null {
  if (cfg.hooks?.enabled !== true) return null;

  const token = cfg.hooks?.token?.trim();
  if (!token) throw new Error("hooks.enabled requires hooks.token");

  const rawPath = cfg.hooks?.path?.trim() || "/hooks";
  if (trimmed === "/") throw new Error("hooks.path may not be '/'");

  return {
    basePath: trimmed,
    token,
    maxBodyBytes: cfg.hooks?.maxBodyBytes ?? 256 * 1024,
    mappings: resolveHookMappings(cfg.hooks),
  };
}
```

认证支持三种方式：

```typescript
// src/gateway/hooks.ts — extractHookToken

export function extractHookToken(req: IncomingMessage): string | undefined {
  // 方式 1：Bearer Token
  const auth = req.headers.authorization;
  if (auth?.toLowerCase().startsWith("bearer ")) {
    return auth.slice(7).trim();
  }

  // 方式 2：自定义请求头
  const headerToken = req.headers["x-openclaw-token"];
  if (headerToken) return headerToken.trim();

  // 方式 3：URL 查询参数（在 HTTP 层处理）
  return undefined;
}
```

方式	示例	说明
Bearer Token	`Authorization: Bearer my-token`	标准 OAuth2 风格

方式	示例	说明
自定义头	`X-OpenClaw-Token: my-token`	不支持 Authorization 头的场景
查询参数	`?token=my-token`	简单场景（注意 URL 日志泄露风险）

两种 Webhook 模式

Webhook 支持两种触发模式：

Wake 模式：注入系统事件

```
POST /hooks/wake
{
  "text": "GitHub 仓库 openclaw/openclaw 收到新 PR #123",
  "mode": "now"
}
```

```
// src/gateway/server/hooks.ts — dispatchWakeHook

const dispatchWakeHook = (value: { text: string; mode: "now" | "next-heartbeat" }) => {
  const sessionKey = resolveMainSessionKeyFromConfig();
  enqueueSystemEvent(value.text, { sessionKey });
  if (value.mode === "now") {
    requestHeartbeatNow({ reason: "hook:wake" });
  }
};
```

Wake 模式直接向主会话注入一条系统事件文本，类似 Cron 的主会话模式。

Agent 模式：启动隔离 Agent

```
POST /hooks/agent
{
  "message": "请审查这个 PR 的代码变更...",
  "name": "GitHub PR Review",
  "channel": "whatsapp",
  "to": "+1234567890",
  "deliver": true
}
```

```
// src/gateway/server/hooks.ts — dispatchAgentHook（简化版）

const dispatchAgentHook = (value) => {
  // 构造一个临时的 CronJob 对象
  const job: CronJob = {
    id: randomUUID(),
    name: value.name,
    schedule: { kind: "at", at: new Date().toISOString() },
    sessionTarget: "isolated",
    wakeMode: value.wakeMode,
    payload: {
      kind: "agentTurn",
      message: value.message,
      model: value.model,
      deliver: value.deliver,
      channel: value.channel,
      to: value.to,
    },
    state: { nextRunAtMs: Date.now() },
  };

  // 异步执行（不阻塞 HTTP 响应）
  void (async () => {
    const result = await runCronIsolatedAgentTurn({ job, message: value.message, ... })

    // 将结果摘要注入主会话
    enqueueSystemEvent(`Hook ${value.name}: ${result.summary}`, { sessionKey });
    if (value.wakeMode === "now") {
      requestHeartbeatNow({ reason: `hook:${jobId}` });
    }
  })();

  return runId;
};
```

Agent 模式复用了 Cron 的隔离执行基础设施——它构造一个临时的 `CronJob` 对象，然后调用 `runCronIsolatedAgentTurn` 执行。**Cron 和 Webhook 共享同一套执行引擎**，区别仅在于触发方式，这是 OpenClaw 里一个很典型的设计取向。

请求体解析

```
// src/gateway/hooks.ts — readJsonBody

export async function readJsonBody(req, maxBytes): Promise<Result> {
  return new Promise((resolve) => {
    let total = 0;
    const chunks: Buffer[] = [];

    req.on("data", (chunk: Buffer) => {
      total += chunk.length;
      if (total > maxBytes) {
        resolve({ ok: false, error: "payload too large" });
        req.destroy();  // 立即断开连接
        return;
      }
      chunks.push(chunk);
    });

    req.on("end", () => {
      const raw = Buffer.concat(chunks).toString("utf-8").trim();
      if (!raw) { resolve({ ok: true, value: {} }); return; }
      try {
        resolve({ ok: true, value: JSON.parse(raw) });
      } catch (err) {
        resolve({ ok: false, error: String(err) });
      }
    });
  });
}
```

注意安全措施：超过 `maxBodyBytes` 时立即 `req.destroy()` 断开连接，防止大负载耗尽内存。

26.3.2 Gmail Pub/Sub 邮件钩子（`src/hooks/gmail.ts`）

概念：邮件驱动的 Agent

Gmail Pub/Sub 集成让 Agent 可以在**收到新邮件时自动处理**——例如自动回复、归档分类、提取关键信息等。

它的工作原理：

```
Gmail 新邮件
    │
    ↓ Google Cloud Pub/Sub 推送
OpenClaw Gateway /hooks/gmail 端点
    │
    ↓ 解析邮件内容
Webhook Agent 模式
    │
    ↓ runCronIsolatedAgentTurn
Agent 处理邮件并投递结果
```

> **衍生解释——Google Cloud Pub/Sub**：Pub/Sub（Publish/Subscribe）是 Google Cloud 的消息队列服务。Gmail API 允许通过 Pub/Sub 监听邮箱变更——当有新邮件时，Google 会向你指定的 HTTPS 端点发送一个推送通知（包含邮件的 `historyId`）。OpenClaw 监听这个推送，然后通过 Gmail API 获取邮件内容。

Gmail 配置

```yaml
hooks:
  enabled: true
  token: "webhook-token"
  gmail:
    account: "user@gmail.com"       # Gmail 账户
    topic: "gog-gmail-watch"        # Pub/Sub 主题
    subscription: "gog-gmail-watch-push"  # Pub/Sub 订阅
    pushToken: "gmail-push-secret"  # Pub/Sub 推送验证令牌
    includeBody: true               # 是否提取邮件正文
    maxBytes: 20000                 # 最大邮件大小（默认 20KB）
    model: "claude-sonnet"          # 可选：为 Gmail 钩子指定模型
    thinking: "low"                 # 可选：思考级别
    allowUnsafeExternalContent: false  # 危险：禁用外部内容安全包装
```

```typescript
// src/hooks/gmail.ts — 默认值

export const DEFAULT_GMAIL_LABEL = "INBOX";
export const DEFAULT_GMAIL_TOPIC = "gog-gmail-watch";
export const DEFAULT_GMAIL_SUBSCRIPTION = "gog-gmail-watch-push";
export const DEFAULT_GMAIL_SERVE_BIND = "127.0.0.1";
export const DEFAULT_GMAIL_SERVE_PORT = 8788;
export const DEFAULT_GMAIL_MAX_BYTES = 20_000;
export const DEFAULT_GMAIL_RENEW_MINUTES = 12 * 60;  // 12 小时续订
```

Webhook URL 构建

Gmail 推送需要一个可访问的 HTTPS URL。OpenClaw 默认使用本地 Gateway 地址：

```typescript
// src/hooks/gmail.ts — buildDefaultHookUrl

export function buildDefaultHookUrl(hooksPath?: string, port = 18789): string {
  const basePath = normalizeHooksPath(hooksPath);
  return `http://127.0.0.1:${port}${basePath}/gmail`;
  // 例：http://127.0.0.1:18789/hooks/gmail
}
```

由于 Google Pub/Sub 要求 HTTPS 端点，本地开发时需要通过 **Tailscale Funnel** 或类似的隧道服务将本地端口暴露为公网 HTTPS 地址。Gmail 配置中的 `tailscale` 字段就是为此设计的。

令牌安全

Gmail 集成涉及两个令牌：

- **`hooks.token`** ——Gateway 的 Webhook 认证令牌
- **`hooks.gmail.pushToken`** ——Pub/Sub 推送验证令牌

```
export function generateHookToken(bytes = 24): string {
  return randomBytes(bytes).toString("hex");  // 48 字符十六进制
}
```

安全审计模块（`src/security/audit-extra.ts`）会检查：

- 令牌长度是否足够（太短会警告）
- `hooks.token` 是否与 Gateway 认证令牌重复（重复会增大攻击面）
- `hooks.path` 是否为 `"/"`（会遮蔽其他 HTTP 端点）

26.3.3 钩子系统总览（`src/hooks/hooks.ts`）

内部钩子 vs 外部钩子

OpenClaw 有两套"钩子"系统，容易混淆：

名称	位置	触发方式	用途
外部钩子（Webhook）	`src/gateway/hooks.ts`	HTTP 请求	外部系统触发 Agent
内部钩子（Internal Hooks）	`src/hooks/hooks.ts`	Agent 生命周期事件	扩展 Agent 行为

前面讨论的是外部钩子。内部钩子是另一套系统——它允许开发者在 Agent 的生命周期中注入自定义逻辑。

内部钩子机制

`src/hooks/hooks.ts` 是内部钩子系统的入口，它重新导出 `internal-hooks.ts` 的所有功能：

```
// src/hooks/hooks.ts

export * from "./internal-hooks.js";

export {
  registerInternalHook as registerHook,
  unregisterInternalHook as unregisterHook,
  clearInternalHooks as clearHooks,
  triggerInternalHook as triggerHook,
  createInternalHookEvent as createHookEvent,
} from "./internal-hooks.js";
```

内部钩子通过事件注册和触发工作：

```
// 注册钩子（在 Agent 启动时）
registerHook("session:new", async (event) => {
  // 当创建新会话时执行
  console.log("新会话创建:", event.sessionId);
});

// 触发钩子（在 Agent 运行时）
await triggerHook("session:new", createHookEvent({
  sessionId: "ses_abc123",
  // ...
}));
```

内置钩子

OpenClaw 自带几个内置钩子（位于 `src/hooks/bundled/` ）：

钩子	目录	功能
`boot-md`	`bundled/boot-md/`	Agent 启动时加载 Markdown 配置文件
`session-memory`	`bundled/session-memory/`	跨会话记忆持久化
`command-logger`	`bundled/command-logger/`	记录用户命令日志
`soul-evil`	`bundled/soul-evil/`	Agent 人格覆盖（实验性）

钩子加载

内部钩子的启用需要配置：

```yaml
hooks:
  internal:
    enabled: true     # 启用内部钩子系统
    entries:          # 逐个钩子启用/禁用
      session-memory:
        enabled: true
      command-logger:
        enabled: false
```

加载过程在 Gateway 启动时执行（ `src/hooks/loader.ts` ），扫描以下位置：

1. **内置钩子目录** —— `src/hooks/bundled/` （随 OpenClaw 分发）
2. **管理目录** —— `~/.openclaw/hooks/` （通过 `openclaw hooks install` 安装）
3. **工作区钩子** —— `<workspace>/hooks/` （项目级钩子）
4. **额外目录** —— `hooks.internal.load.extraDirs` 配置的自定义路径

每个钩子目录必须包含一个 `HOOK.md` 前置数据文件（描述钩子的元数据）和一个 `handler.ts` / `handler.js` 处理函数。

Cron、Webhook 和内部钩子的关系

```
┌──────────────────────────────────────────────────────────────┐
│ 触发源                                                        │
│                                                               │
│  时间调度（Cron）     ──→ CronService ──→ executeJobCore │
│  HTTP 请求（Webhook） ──→ /hooks/*   ──→ dispatchHook   │
│                                          │                    │
│                                  ┌──────────┘                  │
│                                  ↓                            │
│                    runCronIsolatedAgentTurn                   │
│                           （共享执行引擎）                     │
│                                  │                            │
│                                  ↓                            │
│                      Agent 生命周期                           │
│                                  │                            │
│                      内部钩子触发点                            │
│                      ├── session:new                          │
│                      ├── agent:bootstrap                      │
│                      ├── agent:end                            │
│                      └── ...                                  │
└──────────────────────────────────────────────────────────────┘
```

三者的关系：

- **Cron** 和 **Webhook** 是不同的**触发源**，但共享同一个 Agent 执行引擎
- **内部钩子**是 Agent 运行时的**扩展点**，无论 Agent 被 Cron、Webhook 还是用户消息触发，内部钩子都会在相应的生命周期点执行

本节小结

1. **Webhook** 通过 HTTP 端点（默认 `/hooks` ）接收外部请求，支持 Bearer Token、自定义头、查询参数三种认证方式
2. Webhook 有两种模式：**Wake**（注入系统事件到主会话）和 **Agent**（启动隔离 Agent 执行）
3. Agent 模式复用 Cron 的隔离执行基础设施——构造临时 `CronJob` 对象调用 `runCronIsolatedAgentTurn`
4. **Gmail Pub/Sub** 集成让 Agent 在收到新邮件时自动处理，通过 Google Cloud Pub/Sub 推送通知实现

5. Gmail 集成需要 HTTPS 端点，本地开发可通过 **Tailscale Funnel** 暴露
6. **内部钩子**是另一套系统——Agent 生命周期扩展点，支持 `session:new` 、 `agent:bootstrap` 等事件
7. 内置钩子包括会话记忆（ `session-memory ）、命令日志（ `command-logger ）等
8. Cron、Webhook 和内部钩子三者互补：Cron/Webhook 是触发源，内部钩子是 Agent 运行时的扩展点